

★ WEEK 4 · LECTURE

Linear Models & Regularization

The straight line, the factor zoo — and how to tame it

Expanded edition · 38 slides · with three hands-on labs

We start with the simplest model of all — a straight line — then meet the danger of having too many clues, and the elegant fix: putting the model on a budget.

Where We Are Going Today

Week 3 gave us the answer key (labels). Now we build our first real model

1

The straight line (linear regression)

The oldest, simplest prediction rule: draw the best line through the dots.

2

The 'factor zoo' problem

With hundreds of possible clues, plain models overfit to lucky noise.

3

Regularization: Ridge, Lasso, Elastic Net

Give the model a budget so it spends only on clues that truly matter.

By the end you will fit all three models and watch Lasso delete weak clues.

1

PART 1 · THE STRAIGHT LINE

The Straight Line

Linear regression — the simplest model

You've Done This Since School

Fitting a line to dots is something you already understand

Plot your study hours (x) against your test scores (y). The dots roughly trend upward. Drawing the straight line that best follows them — that IS linear regression.

Why start here

- It's the simplest possible model.
- It's fast, and easy to interpret.
- Surprisingly, it often beats fancy models in noisy finance.

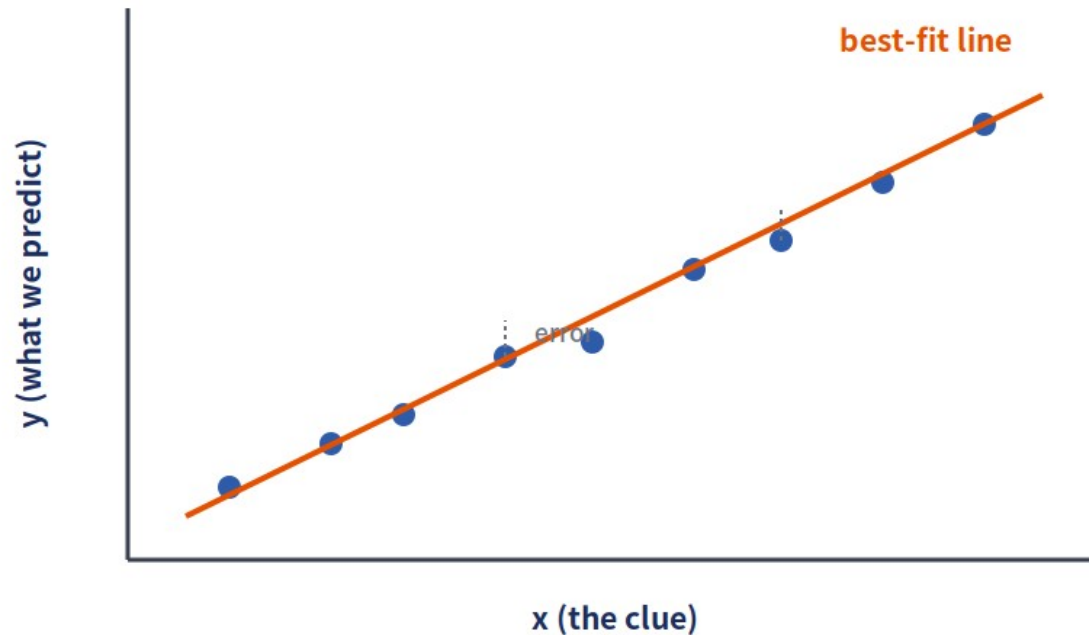
What 'fitting' means

- Find the line closest to all the dots at once.
- The model is just the slope and the height of that line.
- Then use it to predict y for a new x .

Drawing the Best-Fit Line

Find the one line that comes closest to every dot — the smallest total squared gap wins.

Linear Regression = Drawing the Best-Fit Line



The whole idea

Each blue dot is one past example.

We draw the ONE straight line that passes as close as possible to all dots.

"As close as possible" = make the total squared error (gray gaps) as small as we can.

That line is our prediction rule.

The Line, as a Formula

With more than one clue, the line becomes a weighted sum

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p$$

Piece by piece

- $\hat{y}$: our prediction. $x_1 \dots x_p$: the clues — could be hundreds.
- $\beta_1, \beta_2, \dots$: the weight on each clue — how much it counts.
- β_0 : the starting point (intercept).

The key insight

- Big beta = important clue. Near-zero beta = barely matters.
- Learning the model = finding good beta numbers.
- Today's whole story is about controlling those betas.

How We Pick the Betas: Least Squares

'Make the total squared error as small as possible' — this is OLS

$$\min_{\beta} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Reading it

- For each example, take the gap between truth y and guess $\hat{y}$.
- Square it — so + and – gaps don't cancel, and big misses hurt more.
- Add up all squared gaps; choose betas making the total smallest.

The good and the bad

- Good: a clean, exact answer; easy to compute.
- Bad: with many clues, OLS happily fits noise.
- It has no 'self-control' — that's the problem we fix next.

The Slope by Hand — a Four-Day Mini-Beta

Predict a stock's move from the market's move (full version: this week's problem set)

$$\hat{\beta}_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2} \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

	D1	D2	D3	D4	Mean
Market x (%)	-2	0	2	4	1
Stock y (%)	-3	0	2	5	1

The arithmetic

- $\sum (x - \bar{x})(y - \bar{y}) = 26$, $\sum (x - \bar{x})^2 = 20$.
- $\beta_1 = 26/20 = 1.30$ — a 'high-beta' stock.

One covariance, one variance, one division — the workhorse of finance is that small.

Why the Humble Line Often Wins in Finance

Low signal-to-noise punishes complexity — the bias-variance lesson pays off here

The statistical reason

- Financial signal is tiny (Week 1: R^2 often under 1%).
- Fancy models spend their flexibility learning noise → variance.
- The line accepts a little bias and buys huge stability.

The practical reason

- Every beta is a sentence: 'momentum adds 0.4% per unit'.
- Easy to audit, easy to explain to a risk committee.
- When it breaks, you can see WHERE it broke.

In a 55%-hit-rate world, interpretable and stable beats clever and fragile.

2

PART 2 · THE FACTOR ZOO

When Too Many Clues Hurt

The factor zoo and overfitting

The Zoo of 400 Factors

Economist John Cochrane warned of a 'factor zoo' running wild

"We have a zoo of new factors. Most are the result of testing thousands of ideas until something looks significant."

How fakes are born

- Test 400 random 'signals' against past returns.
- By luck alone, ~20 will look impressive (5% of 400).
- Researchers publish the lucky 20; the failures are forgotten — Week 2's survivorship bias, all over again.

Why it matters for us

- Hand all 400 clues to plain OLS and it grabs the fakes.
- The model looks brilliant on history, then loses money live.
- We need a tool that automatically distrusts weak clues: regularization.

The Factor Zoo, Visualized

Hundreds of candidate clues, only a few of them real. The job: keep the real, delete the lucky — automatically.

The "Factor Zoo" — Too Many Suspects

The problem

Researchers have published 400+ "factors" that supposedly predict returns.

Size, value, momentum, weather, sunspots, the Super Bowl winner... it never ends.

If you test 400 random ideas, a handful will look great by pure luck.

Most of the "zoo" is noise in disguise.

What we need

A way to keep the FEW real signals and throw out the many fakes.

If we feed all 400 into a plain model, it overfits to the lucky fakes.

Regularization (today's tool) automatically shrinks or deletes the weak ones.

Lasso can set useless factors to exactly 0.

Too Many Clues = Overfitting (Again)

More features is not always better — often it's worse

Imagine a detective with 400 'suspects' and only 100 clues to go on. They can always invent a story that fits the past perfectly — but it won't catch the next thief.

The symptom

- Near-perfect fit on training data.
- Terrible performance on new data.
- Coefficients become huge and unstable.

The cure (Section 3)

- Don't let the betas grow without limit.
- Add a penalty for using too much 'clue strength'.
- This gently forces simplicity = better on new data.

The Zoo, in Four Numbers

Why 'test everything, keep what works' manufactures fake alpha

400+

Published factor
candidates in the
academic zoo

~20

Expected to look
'significant' by pure
luck (5% of 400)

100

Independent-ish
observations a
detective may really
have

$p > n$

More suspects than
clues — OLS can
'explain' anything

Twenty lucky fakes look exactly like signal — until the model trades them live.

The Tell: Exploding, Flip-Flopping Betas

Finance clues are all cousins — and correlated clues make OLS go wild

What you'll see

- Two near-identical clues (e.g. 6m and 12m momentum) get betas +48 and -47.
- Remove one week of data → the signs flip.
- The fit barely changes; the 'story' changes completely.

Why finance is the worst case

- Value, quality, size, momentum — all measured from overlapping prices and books.
- Correlated columns give OLS infinite ways to split the credit.
- A penalty on beta size ends the credit war — that is Ridge.

Huge, unstable betas are the smoke; overfitting is the fire.

3

PART 3 · REGULARIZATION

Regularization

A budget that forces the model to be simple

Put the Model on a Budget

Limit the total 'clue strength' and the model must spend it wisely. The knob λ sets how strict the budget is.

Regularization = Putting the Model on a Budget

No budget (plain OLS)

"Use as many clues as you like,
as strongly as you like."

The model grabs every clue, including
the lucky fakes from the zoo.

It fits the training data beautifully...
then fails on new data. (Overfitting.)

This is "Student A" from Week 1.

With a budget (regularized)

"You have a limited budget of
total 'clue strength' to spend."

Now the model must spend wisely —
only on clues that truly earn their keep.

Weak clues get shrunk toward zero.
The result is simpler and more robust.

The size of the budget is set by λ .

Ridge Regression (L2 Penalty)

Old goal + a new penalty for big betas

$$\min_{\beta} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p \beta_j^2$$

What changed

- First part: same 'fit the data' as OLS.
- Second part: add up the SQUARES of the betas, times λ .
- Huge betas are now expensive, so the model keeps them small.

What λ does

- $\lambda = 0$: no budget $\rightarrow$ back to plain OLS.
- λ big: very strict $\rightarrow$ all betas shrink toward zero.
- We pick λ using the validation set (Week 1).

Lasso Regression (L1 Penalty)

One small change — absolute value instead of square — does something magical

$$\min_{\beta} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p |\beta_j|$$

The one difference

- Penalize the ABSOLUTE value of betas, not the square.
- This small change has a big effect on the math.

Why that's magical

- Weak betas don't just shrink — they snap to EXACTLY zero.
- A beta of exactly 0 means that clue is DELETED.
- Automatic feature selection — perfect for the factor zoo.

Ridge vs. Lasso vs. Elastic Net

Ridge shrinks all; Lasso deletes the weak; Elastic Net blends both. Pick by the problem.

Ridge vs. Lasso — Two Kinds of Budget

Ridge (L2)	Lasso (L1)	Elastic Net
Shrinks everyone fairly. Penalizes the SQUARE of each coefficient. Keeps ALL clues, but makes each one smaller. Good when many clues each add a little. Nothing becomes exactly zero.	Deletes the useless ones. Penalizes the ABSOLUTE value of each coefficient. Sets weak clues to EXACTLY zero — automatic selection. Good for the factor zoo: it picks the few that matter. Gives a short, readable model.	Best of both. A blend of Ridge + Lasso. Deletes useless clues AND shares credit among similar ones. The practical default when clues overlap.

Three Budgets at a Glance

Same idea — a penalty on the betas — three personalities

Method	Penalty	Effect on betas	Reach for it when...
Ridge (L2)	$\lambda \cdot \sum \beta^2$	All shrink smoothly; none hit zero	Clues are correlated cousins; keep them all, calm them down
Lasso (L1)	$\lambda \cdot \sum \beta $	Weak ones snap to exactly 0	You suspect most clues are fakes — the factor zoo
Elastic Net	mix of both	Shrinks AND deletes	Correlated groups where you still want selection

One λ knob (plus a mix ratio for Elastic Net) — all tuned on the validation set, never the test set.

Choosing λ — Week 1's U-Curve Returns

The budget dial trades bias for variance, exactly like model complexity did

$\lambda = 0$ — no budget

plain OLS

Maximum flexibility. Training error is lowest; the factor zoo eats you alive on test data.

high variance

λ just right

the sweet spot

Found where VALIDATION error bottoms out. Enough freedom for signal, no room for noise.

pick by validation

λ huge — starvation

all betas $\rightarrow 0$

The model can afford nothing and predicts the average for everyone. Underfitting.

high bias

Sweep λ , plot validation error, take the minimum — the same U-shape you drew in Week 1.

Shrink vs. Delete — Three Betas, Two Fates

Watch the same three clues pass through each budget (full arithmetic: this week's problem set)

Clue (standardized)	OLS beta	After Ridge (shrink)	After Lasso, $\lambda = 0.25$ (threshold)
value	0.90	0.72 — smaller, alive	0.65 — survives
momentum	0.35	0.28 — smaller, alive	0.10 — barely survives
'lucky' factor	0.05	0.04 — smaller, alive	0 — DELETED

Ridge lowers every salary; Lasso fires the weakest employees. The zoo needs the second kind of manager.

Scale Your Clues First

The budget compares beta sizes — so the units of your clues rig the contest

The trap

- Market cap in dollars (billions) vs. momentum in % (single digits).
- The cap clue needs a microscopic beta; momentum needs a big one.
- The penalty then punishes momentum for its units, not its merit.

The fix (one line)

- Standardize every clue: subtract its mean, divide by its std.
- Now every beta is in the same currency — the contest is fair.
- sklearn: StandardScaler, or a Pipeline so the test set never leaks.

Rule of thumb: never run Ridge or Lasso on raw, unscaled features.

4

PART 4 · APPLICATION

Where This Shows Up

Index tracking with a handful of stocks

Tracking an Index with Few Stocks

A real Lasso use case you can picture instantly

Goal: copy the S&P 500's returns — but you don't want to buy all 500 stocks. Can you match it closely using only, say, 15 of them? Lasso finds which 15.

How Lasso helps

- Clues (x): the returns of all 500 stocks.
- Target (y): the index's return.
- Lasso sets most stock weights to zero — leaving a small, tradeable basket that tracks the index.

Why this is valuable

- Fewer stocks = lower trading costs.
- Simpler to manage and rebalance.
- Same idea powers many real index funds and ETFs — you'll build a mini version in the lab.

Other Places You'll Meet the Budget

The same shrink-or-delete logic runs quietly through the industry

Setting	How the budget is used
Credit scoring	Lasso trims hundreds of applicant attributes to a scorecard a regulator can read.
Nowcasting GDP	Ridge blends dozens of correlated indicators without letting any one explode.
Hedging a portfolio	Lasso picks a few liquid futures that best offset a messy book.
Combining alpha signals	Elastic Net weights many correlated signals — shrink the family, drop the noise.

Whenever you hear 'we use only the features that matter' — a budget chose them.

Which Tool? — A 60-Second Drill

Cover the right column, answer out loud, then check yourself

Scenario	Which tool?
3 clues, 10,000 rows, clues barely correlated	Plain OLS — no zoo, no need for a budget
400 candidate factors, most probably fake	Lasso — delete the fakes to exactly zero
6m and 12m momentum both in — betas explode	Ridge — calm correlated cousins, keep both
Groups of correlated clues AND you want selection	Elastic Net — shrink the family, drop the noise
Betas of +48 and -47 on near-twin clues	The exploding-beta symptom — any budget fixes it

If you got 4 of 5, the toolbox of Week 4 is yours.

5

PART 5 · HANDS ON

Your Turn — Hands On

Fit all three models and watch Lasso delete weak clues

Getting This Week's Data

Companion notebook — the multi-stock panel is in Section 3(b)

From the companion notebook (§ 3b)

- Open FML_Data_Setup_Guide.ipynb in Colab.
- Run Setup (§ 0), then the § 3(b) panel loader.
- It downloads daily returns for ~10 stocks at once.

What you get back

- `rets`: rows = days, columns = stocks.
- We'll predict one stock (or an index) from the others.
- Offline backup (§ 5) gives a simulated panel.

[CODE] FML_Data_Setup_Guide.ipynb · § 3(b) panel loader

```
import yfinance as yf
tickers = ['AAPL', 'MSFT', 'GOOGL', 'AMZN', 'META', 'KO', 'PEP', 'XOM', 'JPM', 'JNJ']
prices = yf.download(tickers, start='2020-01-01', end='2024-12-31',
                    auto_adjust=True, progress=False)['Close'].dropna()
rets = prices.pct_change().dropna() # rows = days, cols = stocks
```

Hands-On Lab 1 — OLS vs. Ridge vs. Lasso

Predict one stock from the others, three ways

[CODE] colab_lab1.py · data from setup guide § 3b

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error as mse

y = rets['AAPL']
X = rets.drop(columns='AAPL')
Xtr, Xte, ytr, yte = train_test_split(X, y, shuffle=False)

for name, m in [('OLS', LinearRegression()),
                ('Ridge', Ridge(alpha=0.1)),
                ('Lasso', Lasso(alpha=0.001))]:
    m.fit(Xtr, ytr)
    print(name, 'test MSE:', mse(yte, m.predict(Xte)))
```

Your task (step by step)

- Load the panel rets (setup guide § 3b).
- Target y = AAPL; clues X = the other stocks.
- Split into train and test (no shuffle!).
- Fit OLS, Ridge, and Lasso.
- Compare test error of each.
- Q: does a budget beat plain OLS here?

Hands-On Lab 2 — Lasso Picks the Few

Watch Lasso delete weak clues by setting weights to zero

[CODE] colab_lab2.py · data from setup guide § 3b

```
import numpy as np
from sklearn.linear_model import Lasso

for a in [1e-5, 5e-5, 2e-4]:
    m = Lasso(alpha=a).fit(X, y)
    kept = X.columns[np.abs(m.coef_) > 1e-8]
    n_zero = (np.abs(m.coef_) <= 1e-8).sum()
    print(f'alpha={a}: {n_zero} weights = 0,',
          f' kept = {list(kept)}')
```

Your task (step by step)

- Reuse X, y from Lab 1.
- Fit Lasso with a few alpha values.
- Count how many weights are exactly 0.
- Print the stocks Lasso KEPT.
- Q: as the budget tightens (bigger alpha), do more weights vanish?

Hands-On Lab 3 — Claude as Your Coding Copilot

AI is allowed in this course — it explains and extends; you understand and verify

[INPUT] Prompt · Claude (paste after your Lab 1 + 2 code)

You are my machine-learning tutor.

- 1) Explain my Lab 1 loop above, one plain-English sentence per line.
- 2) Build a mini index tracker: y = the equal-weight average of all 10 stocks; X = the 10 stocks. Standardize X with a sklearn Pipeline.
- 3) Sweep Lasso alpha over a log grid; for each alpha record how many stocks are kept and the tracking error (std of $y - \hat{y}$) on a time-ordered test set.
- 4) Plot kept-count and tracking error vs alpha, and print the smallest basket within 1.5x the best error.
- 5) Explain in 3 sentences why Lasso (not Ridge) is the right tool for building a small tracking basket.

House rules for AI use

- Allowed: explain, debug, extend code you wrote.
- Required: run it, read it, defend every line.
- Forbidden: submitting code you cannot explain.
- Tip: ask 'why', not just 'do'.

Comprehension Check

Try each question first — the answer is revealed below

Q1 Which method can set a coefficient to **EXACTLY** zero (delete a clue)?

- A** Plain OLS
- B** Ridge (L2)
- C** Lasso (L1) ✓
- D** None of them

Answer: Lasso's absolute-value penalty snaps weak weights to exactly 0 — automatic feature selection.

Q2 Why is the 'factor zoo' dangerous for plain OLS?

- A** OLS is too slow
- B** OLS overfits to lucky fake factors ✓
- C** OLS deletes good factors
- D** OLS needs Kaggle data

Answer: With hundreds of clues, OLS has no budget and happily fits noise — it grabs the lucky fakes.

Comprehension Check — Round Two

Two more — these cover the new slides

Q3 You crank λ up to a huge value. The betas will:

- A** Explode — more freedom
- B** All shrink toward zero (underfit) ✓
- C** Stay exactly the same
- D** Flip their signs

Answer: A huge budget penalty starves the model: every beta shrinks toward 0 and it predicts the average — underfitting.

Q4 Before running Ridge on cap-in-dollars and momentum-in-%, you should:

- A** Nothing — Ridge handles units
- B** Standardize every feature first ✓
- C** Drop the momentum clue
- D** Square all the features

Answer: The penalty compares beta sizes, so units rig the contest — standardize (mean 0, std 1) before any budget.

The Assembly Line Reaches the Model Station

Weeks 2–3 built the (X, y) table; Week 4 fit the first $f(x)$; Week 5 changes the question

Bars → Events → Labels → Linear $f(x)$ with a budget (today) → Classification (Week 5)

What is now locked in

- A clean (X, y) table from Weeks 2–3.
- A fitted, budgeted linear model — betas you can read aloud.
- A λ chosen on validation data, never on the test set.

What Week 5 changes

- The label becomes a category: default / no default, up / down.
- The line bends into a probability (logistic regression).
- Same budget logic carries over — regularized classifiers.

Three Ideas, One Picture Each

The theme: simpler is safer. A budget on complexity is your best defense against noise

1

Linear model = the best-fit line

Prediction = a weighted sum of clues. Learning = choosing the weights (betas) via least squares.

2

Too many clues = overfitting

The factor zoo: hundreds of candidate signals, mostly lucky noise. Plain OLS grabs the fakes.

3

Regularization = a budget

Ridge shrinks all betas; Lasso deletes the weak (selection); Elastic Net blends both.

Two pictures to keep: the best-fit line, and a model spending a limited budget of clue strength.

Week 4 — Three Things to Remember

Before next class: finish Labs 1–3 and note which stocks Lasso kept for tracking

1

Linear regression is the workhorse

Draw the best-fit line: $\text{prediction} = \text{beta0} + \text{beta1} \cdot x_1 + \dots$ Fit by least squares (OLS).

2

More features can hurt

The factor zoo has 400+ clues, mostly noise. Plain OLS overfits to the lucky fakes.

3

Regularization tames complexity

Ridge (L2) shrinks; Lasso (L1) deletes weak clues to exactly zero; Elastic Net blends both.

Next week — Classification: Logistic Regression, KNN, Naive Bayes (and our first anomaly detector).